

1. Defina as seguintes funções sobre listas em Haskell:

- (a) `swapfl :: [a] -> [a]` : Dada uma lista, troca o primeiro e o último elemento de lugar. Ex.: `swapfl [1,2,3,4]` resulta em `[4,2,3,1]`.
- (b) `maxList :: [Int] -> Int` : Retorna o maior elemento da lista.
- (c) `minList :: [Int] -> Int` : Retorna o menor elemento da lista.
- (d) `split3 :: [a] -> [[a]]` : Dada uma lista, a subdivide em listas com no máximo 3 elementos.
- (e) `splitOn :: Eq a => a -> [a] -> [[a]]` : Divide uma lista com base em um elemento separador. Por exemplo, `splitOn "asdfahjkaqw"` resulta em `["sdf","hjk","qw"]`.
- (f) `intersperse :: a -> [a] -> [a]` : Insere um dado elemento entre cada dois elementos da lista. Exemplo: `intersperse 1 [23,24,25,26]` resulta em `[23,1,24,1,25,1,26]`.
- (g) `merge :: [Int] -> [Int] -> [Int]` : Intercala duas listas ordenadas de inteiros.
- (h) `digits :: Int -> [Int]` : Converte um número inteiro positivo em uma lista de dígitos correspondentes. Ex.: `digits 821` resulta em `[8,2,1]`.
- (i) `digits2int :: [Int] -> Int` : Converte uma lista de dígitos em seu número inteiro positivo correspondente.
- (j) `zip :: [a] -> [b] -> [(a,b)]` : Agrupa duas listas em uma lista de pares. Exemplo: `zip [1,2,3,4] ['a','b','c']` resulta em `[(1,'a'),(2,'b'),(3,'c')]`.

2. Considere a seguinte definição de concatenação:

```
(++) :: [a] -> [a] -> [a]
(++) [] ys = ys
(++) (x:xs) ys = x : (xs ++ ys)
```

Mostre que, para quaisquer listas `xs`, `ys` e `zs`, vale:

$$xs ++ (ys ++ zs) = (xs ++ ys) ++ zs$$

3. Defina a função de ordem superior `until`, que capture o padrão de comportamento de um laço com repetição no final.

4. Defina a função de ordem superior `zipWith :: (a -> b -> c) -> [a] -> [b] -> [c]`, que, dadas duas listas, as agrupa em uma única lista de acordo com a função de transformação provida. Em seguida, mostre que `zip` pode ser expresso por meio de `zipWith`.

5. Defina as seguintes funções em termos de `foldr`:

- (a) `map :: (a -> b) -> [a] -> [b]`
- (b) `filter :: (a -> Bool) -> [a] -> [a]`
- (c) `takeWhile :: (a -> Bool) -> [a] -> [a]`
- (d) `dropWhile :: (a -> Bool) -> [a] -> [a]`
- (e) `maximum :: Ord a => [a] -> a`
- (f) `reverse :: [a] -> [a]`
- (g) `splitOn :: Eq a => a -> [a] -> [[a]]`
- (h) `digits2int :: [Int] -> Int`

6. Expresse `foldl` em termos de `foldr`.